VUB Course Build - Meeting Brief for Alison

Short Version

This was not built in one single course-builder program. It is a static website/course package built with standard web files:

- HTML for pages, slides, handouts, assessments, and instructor guides
- CSS for VUB branding, large accessible text, printable layouts, and responsive screens
- JavaScript for slide navigation, progress tracking, quiz scoring, and report generation
- Google Forms and Google Sheets for centralized assessment submission when needed
- Google Apps Script to create the Forms from the question bank
- Node/npm, Playwright, Python scripts, and Netlify for testing, packaging, PDF generation, and optional hosting
- AI coding assistants were used to draft, organize, refactor, and fill production gaps, with instructor review and edits

The best way to explain it: "We built a lightweight course website that can run locally or online. Google Forms can collect assessment answers, but the slides, handouts, printable resources, and some reports are custom-built web pages."

What Google Forms Does

Google Forms is useful for the assessment collection side:

- Students submit pre-test and post-test answers.
- Responses go to Google Sheets automatically.
- Google Forms quiz mode can score multiple-choice questions.
- The response Sheet can be filtered, exported, or summarized.
- The Forms can be embedded inside course pages so students do not need to hunt for links.

For the Financial Readiness course, the setup is documented in:

- `VUB Financial Readiness Course/📘 Assessments/SETUP-GOOGLE-FORMS.md`
- `VUB Financial Readiness Course/scripts/google-forms-setup.gs`
- `VUB Financial Readiness Course/📘 Assessments/submit-tests.html`
- `VUB Financial Readiness Course/📘 Assessments/pre-test-form.html`

- `VUB Financial Readiness Course/📘 Assessments/post-test-form.html`

The Apps Script creates:

- A Drive folder for test materials
- A Pre-Test Google Form
- A Post-Test Google Form
- Quiz questions with answer keys
- Student-facing Form URLs
- Edit URLs for the instructor

What The Report Portion Is

There are two report approaches in the current work.

1. Browser-Based Individual PDF Report

This is used in the Intermediate Computer Skills assessments.

How it works:

- The assessment is an HTML page with questions stored in JavaScript.
- When the student submits, JavaScript calculates the score.
- It breaks the score down by category.
- The pre-test result is saved in the browser's localStorage on that same computer.
- The post-test reads the saved pre-test score and compares growth.
- A hidden report section is filled in automatically.
- The Print Report button calls the browser print dialog.
- The report can be printed or saved as a PDF.

Key files:

- `VUB Intermediate Computer Course/weeks/week-01/pre-test.html`
- `VUB Intermediate Computer Course/weeks/week-08/post-test.html`

Strengths:

- Works offline.
- Produces a polished individual student report immediately.
- Does not require a Google login.
- No student data leaves the computer unless the report is printed or saved.

Limitations:

- It is not centralized.
- localStorage is tied to the same browser and computer.

- It is not ideal for statewide aggregate reporting unless results are collected separately.

2. Google Forms + Sheets Reporting

This is the better approach if Alison wants to recreate reports for other areas of the state.

How it works:

- Students submit through Google Forms.
- The responses land in Google Sheets.
- The Sheet becomes the source of truth for reports.
- Reports can be built using Google Sheets charts, pivot tables, filters, or Looker Studio.
- Pre-test and post-test rows can be compared by student name or another consistent identifier.

Strengths:

- Centralized.
- Easier for multiple locations.
- Exportable.
- Easier to share with leadership.
- Easier to preserve for grant reporting or program evaluation.

Limitations:

- Requires internet access.
- Requires Form and Sheet ownership/permissions to be managed.
- Needs a clear privacy policy before collecting names or other identifiers.

Course Development Process

1. Discovery

We started with practical questions:

- Who is the audience?
- What do they already know?
- What do they need to do after class?
- What does success look like?
- What has to work offline?
- What should be measured before and after instruction?

For Intermediate Computer Skills, the audience was older veterans who had completed a basic computer course. The course was designed to be slow-paced, hands-on, readable, and confidence-building.

2. Curriculum Map

The course was organized by week or module:

- Learning objectives
- Class timing
- Slide flow
- Hands-on practice
- Handouts
- Instructor notes
- Assessment questions
- Pre/post comparison categories

The Intermediate Computer Skills course uses an 8-week structure. The Financial Readiness course uses a module-based structure with weekly folders.

3. Design System

The VUB style was standardized:

- Navy, red, gold, white/off-white palette
- Large readable text
- High contrast
- Keyboard-accessible navigation
- Printable handouts
- Consistent card, button, header, and footer patterns

The goal was not just to look polished. It was to reduce cognitive load for older learners.

4. Build

Most resources are plain files:

- `index.html` for the course launcher
- `financial-readiness.html` and `intermediate-computer-skills.html` for course pages
- `weeks/week-XX/presentation.html` for weekly slide decks
- `syllabus.html` files for instructor guides
- handout HTML/PDF files for classroom support
- assessment HTML/Form files for pre-tests and post-tests

The static website approach was chosen because it is portable, inexpensive, and easy to host.

5. Review And Verification

We checked for:

- Broken links

- Readability
- Browser behavior
- Print layout
- Assessment flow
- Offline backup
- Classroom usability

The Financial Readiness course also has Playwright tests and build scripts:

- `npm run build:site`
- `npm test`
- `npm run package:travel`

Tools And Workarounds

Static Website Instead Of LMS

Gap: No need for a full learning management system.

Workaround: Build a static course site that opens in a browser, can be hosted on Netlify, and can also run from a flash drive.

Google Forms For Central Collection

Gap: Custom HTML assessments create nice reports but do not centrally collect results.

Workaround: Use Google Forms and Sheets for statewide or multi-location collection.

Browser Print To PDF

Gap: Need a clean individual report without buying reporting software.

Workaround: Build an HTML report section and use browser print/save-as-PDF.

Apps Script For Repeatability

Gap: Manually creating identical pre-test and post-test Forms is slow and error-prone.

Workaround: Use Google Apps Script to generate the Forms from a question list.

Travel Package

Gap: Satellite classrooms may have weak internet or different equipment.

Workaround: Create a flash-drive-ready package and zip backup with `npm run package:travel`.

AI Assistance

Gap: Building slides, handouts, assessments, printable versions, answer keys, and instructor guides by hand is time-consuming.

Workaround: Use AI coding assistants to draft and assemble materials quickly, then review, correct, and adapt them for the actual class.

Best Recommendation For Alison

If Alison wants to recreate reports for other areas of the state, recommend this path:

1. Use Google Forms for pre-test and post-test collection.
2. Link each Form to a Google Sheet.
3. Use consistent student identifiers, such as full name plus class location.
4. Keep the same categories across pre-test and post-test.
5. Build a Google Sheets summary tab or Looker Studio dashboard.
6. Use the static course website only as the student-facing launchpad.
7. Keep PDF/printable reports optional for individual student feedback.

This gives her centralized reporting without requiring every site to manage custom HTML report files.

Questions To Ask Alison

- Does she need individual student reports, aggregate site reports, or both?
- Does she need reports by class location, instructor, county, cohort, or date?
- Will students have reliable internet during assessment time?
- Who should own the Google Forms and Sheets?
- Who should be able to view raw student responses?
- Does she need names, anonymous IDs, or both?
- Does she need pre/post improvement by category?
- Does she need reports exported to PDF for leadership?
- Does the state already use Google Workspace, Microsoft 365, or another reporting platform?

Suggested Meeting Agenda

1. Start with the big picture: "This is a static course website plus optional Google Forms reporting."
2. Show the course homepage and one lesson.
3. Show a handout and instructor guide.
4. Show the Intermediate custom pre/post report flow.
5. Show the Financial Readiness Google Forms setup flow.
6. Discuss which reporting model fits her statewide use case.
7. Agree on next steps: clone a template, define questions/categories, decide who owns the Forms, and build a sample report.

Plain-English Explanation To Use

"The lessons themselves are built like a small website. Each course has HTML pages for the slides, handouts, assessments, and teacher guides. That keeps it portable: we can run it from a folder, a flash drive, or a simple website host. For assessments, we used two approaches. The computer skills course has custom browser-based reports that can print to PDF. The financial readiness course also has a Google Forms workflow, where responses go into Google Sheets. If you want statewide reporting, I would use the Google Forms and Sheets path, then build summaries from the Sheet."

Follow-Up Items To Offer

- Share the Google Forms setup document.
- Share the Apps Script template.
- Share one sample pre/post question bank.
- Build a small proof-of-concept report for one other program area.
- Create a reusable template folder for future VUB courses.